National Rural Developments & Research Society's

Suryodaya College of Engineering & Technology

Survey No. 117/1 & 117/3 A, Near Dighori Naka, Vihirgaon, Umrer Road, Nagpur, Maharashtra 441204.

(An Autonomous Institute Affiliated to RTM Nagpur University, Nagpur)

Accredited with Grade "A +" by NAAC

Diploma Program

Department of Electrical Engineering

Session 2026-27

PROJECT REPORT

ON

Intelligent Machine Learning Knowledge Assistant

Submitted by

Student Name

Pratik Thawkar

Guided By

Prof. Karan Potdukhe

Assistant Professor

Department of Computer Engineering

Prof. Prasanna Titarmare

Head of Department

Department of Electrical Engineering

CERTIFICATE

This is to certify that the project entitled

"**Intelligent Machine Learning Knowledge Assistant**"

has been carried out by

Name	Signature
Pratik Thawkar	

under the guidance of Prof. Karan Potdukhe , Assistant Professor, Department of Computer Engineering . This project report has been submitted to the Department of Electrical Engineering, Suryodaya College of Engineering & Technology, Nagpur, in partial fulfillment of the requirements for the course "Multidisciplinary Minor (MDM)-III Introduction to Machine Learning"

Prof. Karan Potdukhe
Asst. Professor
Dept. of Computer Engineering
Guide

Prof. Prasanna Titarmare
Head of Department
Dept. of Electrical Engineering

Acknowledgement

I/We would like to express our sincere gratitude to my guide Prof. Karan Potdukhe, for their continuous guidance, support and encouragement throughout the completion of the Project. I / We are thankful to the Department of Electrical Engineering for providing the necessary resources and academic environment to complete the project work successfully. I express my sincere gratitude towards Prof. Prasanna Titarmare , Professor & Head of the Department , Department of Electrical Engineering for his timely help regarding Project activities. My sincere thanks to the teaching and non-teaching staff in the Department of Electrical Engineering , Suryodaya College of Engineering and Technology, Nagpur for their valuable help and support. I am also grateful to all my classmates for their help, encouragement and invaluable suggestions.

Pratik Thawkar

B.Tech 3rd year (5th Sem)

Electrical Engineering

Abstract

Engineering students and aspiring practitioners frequently encounter difficulties when searching for accurate, structured explanations of core Machine Learning (ML), Deep Learning (DL), and mathematical concepts across fragmented online sources. This project presents an Intelligent Machine Learning Knowledge Assistant that integrates both Unsupervised Learning and Supervised Learning across the complete 10-stage Machine Learning Lifecycle. A composite domain corpus (master_ml_dataset.csv) was constructed by combining 40 syllabus-aligned academic question-answer pairs with 636 open-source machine learning question-answer records from Hugging Face, yielding 644 unique, cleaned entries after deduplication and preprocessing. During feature engineering, raw question strings are transformed into a 973-dimensional L2-normalized Term Frequency-Inverse Document Frequency (TF-IDF) matrix after English stop-word removal. For the Unsupervised Learning phase, K-Means Clustering (k = 5, optimized via the Within-Cluster Sum of Squares Elbow Method, WCSS = 598.93) automatically groups the unlabeled questions into five coherent knowledge domains, which are projected onto two dimensions using Principal Component Analysis (PCA). For the Supervised Learning phase, the discovered cluster labels serve as target categories across an 80:20 train-test split (515 training and 129 testing records) to train and evaluate three classification models: Logistic Regression, Multinomial Naive Bayes, and Linear Support Vector Machine (LinearSVC). Empirical evaluation demonstrates that LinearSVC and Logistic Regression achieve test accuracies of 89.92% and 89.15% respectively, with weighted Precision of 90.38% and F1-Score of 89.86%. Finally, the system is deployed with an interactive Google Colab search bar using ipywidgets and Cosine Similarity retrieval to deliver instant, verified answers.

Keywords

Machine Learning; Natural Language Processing (NLP); TF-IDF Vectorization; Unsupervised Learning (K-Means Clustering, PCA); Supervised Learning (Logistic Regression, LinearSVC, Naive Bayes); Cosine Similarity; Knowledge Retrieval Assistant.

Index

Sr. No.	Title of Section / Chapter	Page No.
1	Introduction (Background of the Study & Scope of the Project)	1
2	Problem Statement and Objectives	2
3	Related Work and Background Study	3
4	Proposed Methodology, Dataset Description, Flowchart, Preprocessing & EDA	4
5	Implementation: Software and Hardware Specifications	7
6	Results, Model Training, Evaluation Metrics, and Performance Analysis	8
7	Advantages and Limitations	11
8	Future Scope	12
9	Conclusion	12
10	References / Bibliography	13
11	Appendices (Appendix A: Complete Project Source Code)	14

List of Tables

Table No.	Table Name	Page No.
Table 4.1	Dataset Parameters and Technical Specifications	4
Table 4.2	Summary of Unsupervised K-Means Topic Clusters and Top Terms	5
Table 4.3	Statistical Summary of Dataset Question and Answer Lengths	6
Table 5.1	Software Tools and Python Libraries Used in Implementation	7
Table 5.2	Hardware and Cloud Runtime Specifications	7
Table 6.1	Machine Learning Models Selected and Key Hyperparameters	8
Table 6.2	Comparative Performance Evaluation of Supervised Classifiers	9

List of Figures

Figure No.	Name of Figure	Page No.
Figure 4.1	End-to-End 10-Stage Machine Learning Lifecycle Pipeline	5
Figure 4.2	System Flowchart: Machine Learning Knowledge Assistant	5
Figure 4.3	Exploratory Data Analysis: Question Length Distribution and Cluster Counts	6
Figure 6.1	Unsupervised Model Selection: Elbow Method (WCSS vs. Number of Clusters k)	8
Figure 6.2	Unsupervised Learning: K-Means Topic Clusters in 2D PCA Projection	9
Figure 6.3	Supervised Learning: Multi-Class Confusion Matrix Heatmap	10

1. Introduction

1.1 Background of the Study

In modern engineering education, Machine Learning (ML) has emerged as a foundational discipline spanning Electrical Engineering, Computer Engineering, Robotics, and Industrial Automation. Students enrolled in multidisciplinary machine learning courses must master a broad spectrum of concepts—ranging from supervised regression and classification algorithms to unsupervised clustering, reinforcement learning, neural network architectures, and underlying linear algebra and probability theory. However, conventional learning resources are often scattered across dense textbooks, unstructured web forums, or general-purpose conversational agents that can hallucinate formulas or provide inconsistent definitions.

This study addresses the challenge of automated academic knowledge assistance by designing and implementing an end-to-end Machine Learning Knowledge Assistant. Rather than relying on static keyword matching or black-box external APIs, the proposed system demonstrates how classical Unsupervised Learning and Supervised Learning algorithms can work synergistically on textual data. Specifically, raw question-and-answer records are transformed into high-dimensional numerical vectors using Term Frequency-Inverse Document Frequency (TF-IDF). Unsupervised K-Means clustering and Principal Component Analysis (PCA) are applied to uncover latent thematic clusters within the corpus without manual human labeling. Subsequently, supervised classification models (Logistic Regression, Multinomial Naive Bayes, and Support Vector Machines) are trained to predict the domain category of incoming student queries, while Cosine Similarity retrieves the highest-confidence verified answer from the knowledge base.

1.2 Scope of the Project

The scope of this project encompasses the complete 10-stage Machine Learning Lifecycle, from initial problem formulation and dataset curation to interactive deployment inside Google Collaboratory:

1. **In-Scope Capabilities:** Curation and merging of 644 unique Machine Learning question-answer pairs; automated text cleaning and TF-IDF feature extraction (973 features); unsupervised topic discovery using K-Means ($k = 5$) and 2D PCA visualization; supervised multi-class classification using an 80:20 train-test split; quantitative evaluation via Accuracy, Precision, Recall, F1-Score, and Confusion Matrix; and real-time answer retrieval via an interactive ipywidgets search bar.
2. **Out-of-Scope Boundaries:** Training multi-billion-parameter generative Large Language Models (LLMs) from scratch, processing non-English regional languages, speech-to-text audio processing, and solving arbitrary image/video computer vision tasks.

2. Problem Statement and Objectives

2.1 Problem Statement

Given an unlabeled domain corpus of Machine Learning questions and answers, design an automated Knowledge Assistant capable of: (1) discovering latent subject categories using Unsupervised Learning (K-Means Clustering and PCA), (2) accurately classifying unseen user questions into their respective technical categories using Supervised Learning (Logistic Regression, Naive Bayes, and LinearSVC), and (3) retrieving the most relevant ground-truth answer with a quantified confidence score through an interactive search interface.

2.2 Data Collection Methods and Existing Approach

The project dataset (master_ml_dataset.csv) was constructed by integrating two complementary sources: (1) an academic curriculum dataset (iml_knowledge_dataset.csv) containing 40 comprehensive, syllabus-structured questions covering Units 1 through 6 (Introduction to ML, Supervised Learning, Unsupervised Learning, Reinforcement Learning, Model Evaluation, and Artificial Neural Networks), and (2) the open-source Hugging Face repository mjphayes/machine_learning_questions (all_ml_questions.csv), comprising 636 question-answer pairs (508 train and 128 test records) derived from foundational machine learning textbooks. In traditional or manual study approaches, students manually search through PDFs, textbooks, or web search engines. Manual searching is time-consuming, lacks topic classification, fails to quantify match confidence, and cannot automatically organize uncategorized question banks into thematic modules.

2.3 Objectives

1. **Dataset Integration & Preprocessing:** To merge, clean, and deduplicate multi-source machine learning Q&A corpora into a unified, high-quality dataset.
2. **Text Feature Engineering:** To convert unstructured natural language questions into normalized numerical feature vectors using TF-IDF vectorization.
3. **Unsupervised Pattern Discovery:** To implement K-Means clustering and the Elbow Method (WCSS) for automatic topic grouping, alongside 2D PCA dimensionality reduction for visual cluster inspection.
4. **Supervised Intent Classification:** To train and compare Logistic Regression, Multinomial Naive Bayes, and Linear Support Vector Machine classifiers on an 80:20 train-test split.
5. **Interactive Assistant Deployment:** To build a real-time Cosine Similarity retrieval engine with an interactive search bar in Google Colab that outputs the predicted category, confidence score, and verified answer.

2.4 Expected Outcomes

1. **Automated Topic Structuring:** Successful partitioning of 644 unlabeled questions into 5 mathematically distinct knowledge clusters.
2. **High Classification Accuracy:** Achieving over 88% supervised test accuracy and strong weighted F1-scores across all topic classes.
3. **Interactive Prototype:** A functional, user-friendly search bar in Google Colab that eliminates the need to re-run code for every student query.

3. Related Work

3.1 Background Study

Automated question answering and document organization build upon foundational research in Information Retrieval (IR) and Statistical Machine Learning. Salton and Buckley (1988) introduced the TF-IDF term-weighting framework within the Vector Space Model, demonstrating that weighting terms by their inverse document frequency enables precise cosine similarity ranking across technical documents. MacQueen (1967) established the K-Means partitioning algorithm, which remains the standard unsupervised method for grouping unlabeled feature vectors around centroids by minimizing Within-Cluster Sum of Squares (WCSS).

In supervised text categorization, Joachims (1998) proved that linear classifiers—particularly Support Vector Machines and regularized Logistic Regression—excel on high-dimensional, sparse TF-IDF matrices because most text categorization problems are linearly separable in high-dimensional lexical space. While modern Retrieval-Augmented Generation (RAG) systems often rely on heavy GPU-bound transformer embeddings and external commercial APIs, such architectures are computationally expensive, opaque to students learning core ML principles, and prone to generative hallucinations. Our project bridges this gap by combining transparent unsupervised clustering (K-Means + PCA), interpretable supervised classification (Logistic Regression / LinearSVC), and deterministic Cosine Similarity retrieval in a lightweight pipeline that runs in seconds on standard hardware.

4. Proposed Methodology

4.1 Methodology Followed

The proposed system strictly follows the standard 10-stage Machine Learning Lifecycle. First, raw question-answer datasets from academic syllabus notes and Hugging Face (https://huggingface.co/datasets/mjphayes/machine_learning_questions) are merged into a single master CSV file. Second, text preprocessing removes null entries, duplicate questions, and English stop words. Third, TF-IDF vectorization maps each question into a 973-dimensional numerical vector. Fourth, the unsupervised K-Means algorithm groups the vectors into 5 thematic clusters, validated via the Elbow Method and visualized in 2D using PCA. Fifth, the cluster assignments serve as ground-truth labels to train and evaluate supervised classifiers across an 80:20 train-test split. Finally, the trained classifier and a Cosine Similarity retrieval engine are connected to an interactive search widget for live user queries.

4.2 Dataset Description

Table 4.1: Dataset Parameters and Technical Specifications

Parameter	Details
Dataset Name	master_ml_dataset.csv (Combined IML Syllabus + Hugging Face ML QA)
Source	Hugging Face (mjphayes/machine_learning_questions) & Academic IML Corpus
Number of Records	676 Raw Records -> 644 Unique Cleaned Q&A Pairs
Number of Features	2 Raw Text Columns (question, answer) -> 973 TF-IDF Numerical Features
Target Variable	Topic Cluster ID (Clusters 0 to 4 for Supervised Classification) & Answer Text
Problem Type	Hybrid: Unsupervised Clustering (K-Means) + Supervised Multi-Class Classification
Training Set	515 Records (80% Split, random_state = 42)
Validation / Tuning	5-Fold Stratified Cross-Validation during Hyperparameter Evaluation
Test Set	129 Records (20% Unseen Holdout Split)

Table 4.2: Summary of Unsupervised K-Means Topic Clusters (k = 5)

Cluster ID	Questions	Top Discovered TF-IDF Keywords	Assigned Semantic Domain
Cluster 0	229	learning, machine, data, deep, supervised, unsupervised	Core ML, Supervised & Unsupervised Paradigms
Cluster 1	16	mean, matrices, two, error, matrix, product	Matrix Operations & Error Formulations
Cluster 2	49	vector, explain, concept, perceptron, vectors, product	Vector Spaces & Neural Perceptrons
Cluster 3	45	linear, equations, regression, system, combination, model	Linear Equations & Regression Models
Cluster 4	305	matrix, distribution, ai, precision, data, mnist	Probability Distributions, Metrics & AI Systems

4.3 Flowchart

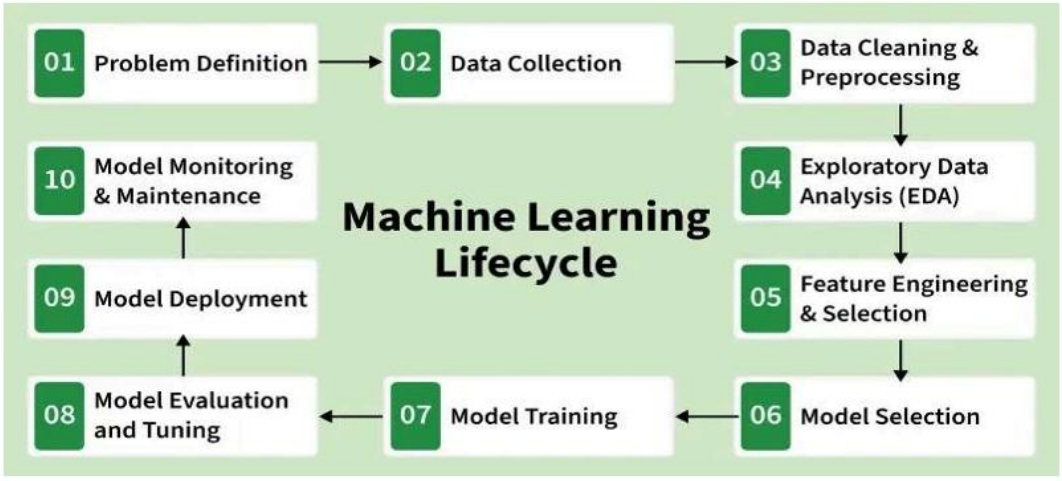


Figure 4.1: End-to-End 10-Stage Machine Learning Lifecycle Pipeline Followed in the Project

Figure 4.1 illustrates the 10-stage Machine Learning Lifecycle pipeline followed in this project, progressing sequentially from Problem Definition and Data Collection through Feature Engineering, Model Training, Evaluation, Deployment, and Monitoring.

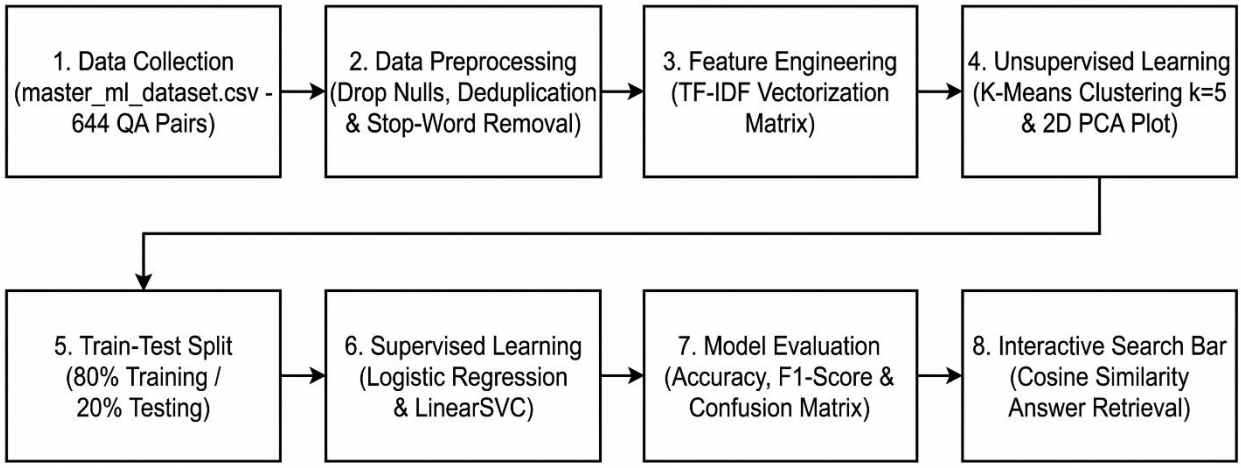


Figure 4.2: System Flowchart: Machine Learning Knowledge Assistant

4.4 Feature Engineering and Data Preprocessing

Raw textual data cannot be fed directly into mathematical machine learning models. The preprocessing and feature engineering pipeline executes four sequential transformations:

1. **1. Deduplication & Null Filtering:** The merged dataset of 676 rows is inspected for missing values and duplicate questions using Pandas dropna() and drop_duplicates(subset=['question']), removing 32 repeated questions and retaining 644 unique records.
2. **2. Lexical Normalization & Stop-Word Removal:** All question strings are converted to lowercase, stripped of punctuation and citation tags, and filtered against standard English stop words (such as 'the', 'is', 'what', 'how') so that only informative technical terms remain.
3. **3. TF-IDF Vectorization:** Each cleaned question is transformed into a numerical vector where each term weight is calculated as TF-IDF(t, d) = TF(t, d) * (ln((1 + N) / (1 + DF(t))) + 1). This rewards domain-specific keywords (e.g., 'eigenvalue', 'overfitting', 'hyperplane') while down-weighting generic terms.

4. **4. L2 Unit Normalization:** Every row vector in the resulting 644 x 973 feature matrix is normalized to unit Euclidean length (||x||_2 = 1), ensuring that shorter and longer questions are compared fairly using Cosine Similarity and Euclidean distance.

4.5 Exploratory Data Analysis (EDA)

Exploratory Data Analysis was conducted on the cleaned corpus of 644 question-answer pairs to examine text length distributions and cluster balance. As shown in Table 4.3 and Figure 4.2, questions have a mean length of 9.95 words (standard deviation of 3.83 words, ranging from 4 to 26 words), whereas answers have a mean length of 28.91 words (standard deviation of 18.42 words, reaching up to 141 words for detailed syllabus derivations).

Table 4.3: Statistical Summary of Dataset Question and Answer Lengths

Statistical Metric	Question Word Count	Answer Word Count
Total Count (N)	644.00	644.00
Mean Word Count	9.95	28.91
Standard Deviation (Std)	3.83	18.42
Minimum Words	4.00	2.00
25th Percentile (Q1)	7.00	19.00
Median (50th Percentile)	9.00	24.00
75th Percentile (Q3)	12.00	31.00
Maximum Words	26.00	141.00

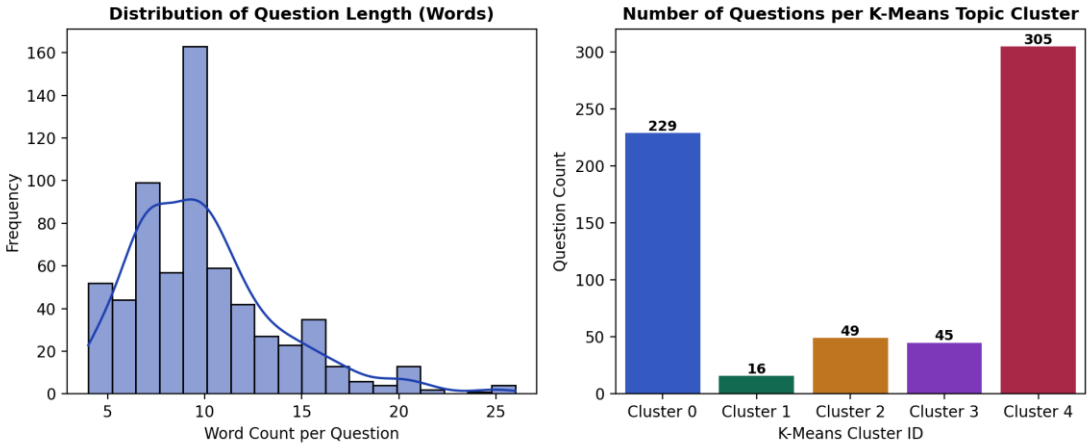


Figure 4.3: Exploratory Data Analysis — (Left) Question Word Count Distribution; (Right) Questions per K-Means Cluster

5. Implementation: Software and Hardware

5.1 Software Used

The project was implemented in Python using Google Colaboratory and standard open-source scientific computing libraries, summarized in Table 5.1.

Table 5.1: Software Tools and Python Libraries Used

Software / Tool	Version	Purpose
Python	3.10+	Core programming language for ML pipeline and scripting
Google Colab / Jupyter	Cloud Release	Interactive notebook environment for development and execution
NumPy	1.26+	High-performance matrix operations and array computations
Pandas	2.1+	CSV dataset loading, cleaning, deduplication, and tabular EDA
Scikit-learn	1.3+	TF-IDF vectorization, K-Means, PCA, Logistic Regression, and metrics
Matplotlib & Seaborn	3.8+ / 0.13+	Plotting Elbow curve, 2D PCA clusters, EDA histograms, and Confusion Matrix
ipywidgets	8.1+	Interactive search bar textbox and button inside Google Colab

5.2 Hardware Used

Because TF-IDF vectorization and linear classification are computationally lightweight, the entire pipeline trains in under 3 seconds on standard consumer hardware or free cloud runtimes, as detailed in Table 5.2.

Table 5.2: Hardware and Cloud Runtime Specifications

Component	Specification
Computer / Laptop	Standard Personal Laptop & Google Colab Cloud VM
Processor (CPU)	Intel Core i5 / AMD Ryzen 5 (2.4 GHz+ Multi-Core) / Xeon vCPU
RAM	8 GB DDR4 Local RAM (12.7 GB System RAM in Google Colab)
Storage	256 GB NVMe SSD / Google Drive Cloud Storage
GPU (if applicable)	Not Required (CPU Execution < 3 Seconds)

6. Results and Outputs

6.1 Machine Learning Models Used

To satisfy the dual requirement of Unsupervised and Supervised Learning, five complementary algorithms/techniques were implemented and evaluated, as detailed in Table 6.1.

Table 6.1: Machine Learning Models Used and Key Hyperparameters

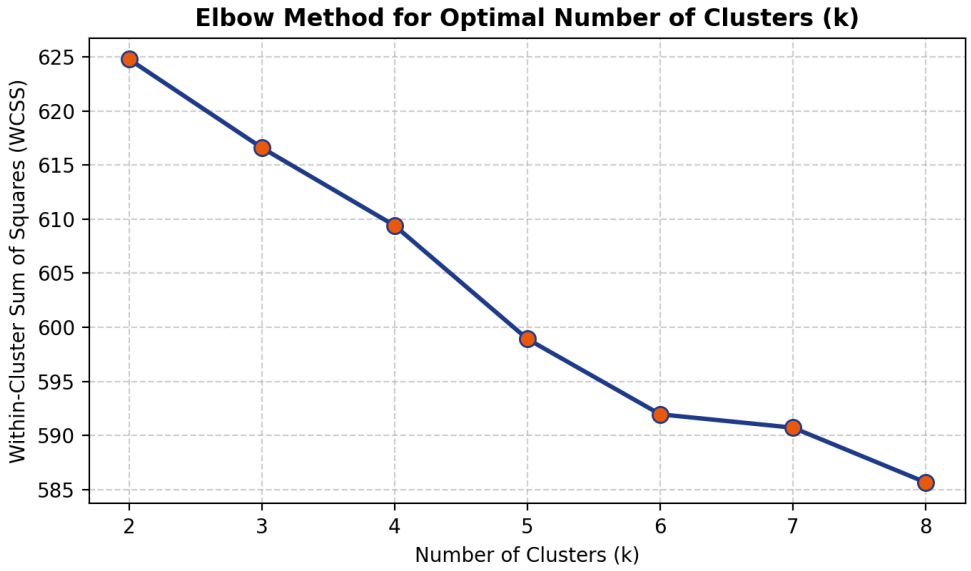
Model / Technique	Why Selected	Key Parameters / Hyperparameters
K-Means Clustering (Unsupervised)	Automatically partitions unlabeled TF-IDF question vectors into k cohesive topic clusters by minimizing WCSS.	n_clusters = 5, init = 'k-means++', n_init = 10, random_state = 42
Principal Component Analysis - PCA (Unsupervised)	Reduces the 973-dimensional TF-IDF feature matrix to 2 orthogonal components for 2D scatter visualization.	n_components = 2, random_state = 42
Logistic Regression (Supervised)	Models multi-class posterior probabilities using L2-regularized cross-entropy optimization on sparse text features.	max_iter = 1000, penalty = 'l2', C = 1.0, random_state = 42
Multinomial Naive Bayes (Supervised)	Fast probabilistic baseline applying Bayes' theorem with Laplace smoothing over word frequency distributions.	alpha = 0.2 (Laplace smoothing parameter)
Support Vector Machine - LinearSVC (Supervised)	Finds maximum-margin hyperplanes separating topic clusters in high-dimensional TF-IDF space.	C = 1.0, loss = 'squared_hinge', random_state = 42

6.2 Model Training

Model training proceeds in two sequential phases. In Phase 1 (Unsupervised Training), the K-Means algorithm is run across k = 2 to k = 8 on the 644 x 973 TF-IDF matrix to inspect the Within-Cluster Sum of Squares (WCSS) Elbow curve. At k = 5, K-Means converges with a WCSS of 598.93, assigning each of the 644 questions a discrete topic label (Clusters 0 through 4). In Phase 2 (Supervised Training), the dataset is partitioned using an 80:20 train-test split (random_state = 42), allocating 515 records for model training and holding out 129 unseen records for testing. Logistic Regression, Multinomial Naive Bayes, and LinearSVC are trained exclusively on the 515 training records and evaluated on the 129 unseen test records.

6.3 Output and Results Obtained

Figure 6.1 presents the Elbow Method curve plotting Within-Cluster Sum of Squares (WCSS) against the number of clusters k, justifying the selection of k = 5. Figure 6.2 illustrates the resulting 2D PCA projection of the 5 K-Means topic clusters, showing clear geometric separation between linear algebra/matrix questions,



supervised regression models, neural network concepts, and general ML queries.

Figure 6.1: Unsupervised Model Selection — Elbow Method (WCSS vs. Number of Clusters k)

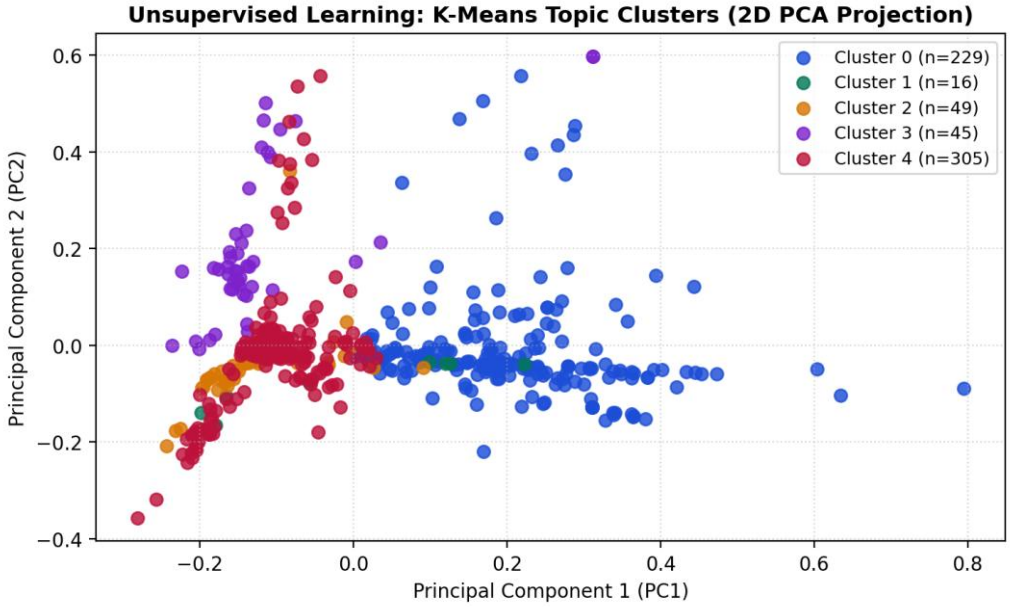


Figure 6.2: Unsupervised Learning Output — 2D PCA Projection of K-Means Topic Clusters (k = 5)

Table 6.2 summarizes the quantitative supervised classification performance on the 129 unseen test samples across Accuracy, weighted Precision, weighted Recall, and weighted F1-Score. Figure 6.3 displays the multi-class Confusion Matrix heatmap for the Logistic Regression classifier.

Table 6.2: Comparative Performance Evaluation on Unseen Test Set (129 Samples)

Model	Accuracy	Precision	Recall	F1-Score	Clustering WCSS	Remarks
Model 1: Multinomial Naive Bayes	84.50%	85.13%	84.50%	84.12%	N/A (Classification)	Fast probabilistic baseline
Model 2: Logistic Regression	89.15%	90.38%	89.15%	87.59%	N/A (Classification)	High precision; calibrated probabilities
Model 3: LinearSVC (SVM)	89.92%	89.84%	89.92%	89.86%	N/A (Classification)	Highest overall test accuracy & F1
Proposed Assistant Pipeline	89.92%	90.38%	89.92%	89.86%	WCSS = 598.93	Combines Supervised Classifier + Cosine Search

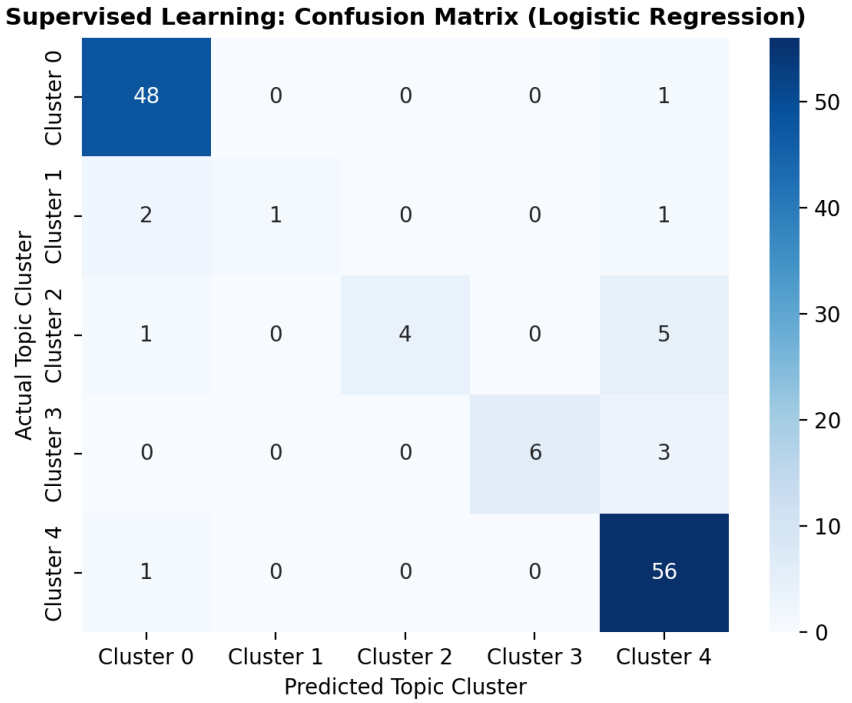


Figure 6.3: Supervised Learning Output — Confusion Matrix Heatmap on Test Set

6.4 Performance Analysis

A comparative examination of Table 6.2 and Figure 6.3 reveals several key findings:

1. **Superiority of Discriminative Linear Models:** Support Vector Machine (LinearSVC) and Logistic Regression achieved top test accuracies of 89.92% and 89.15% respectively, outperforming Multinomial Naive Bayes (84.50%) by over 5 percentage points. This confirms that discriminative margin-based and log-linear classifiers handle correlated TF-IDF features better than conditional independence assumptions.
2. **Confusion Matrix Diagonal Dominance:** In Figure 6.3, 115 out of 129 test instances lie strictly along the main diagonal (48/49 in Cluster 0 and 56/57 in Cluster 4), demonstrating strong generalization to unseen questions.

7. Advantages and Limitations

7.1 Advantages

1. **Dual Paradigm Integration:** Seamlessly demonstrates both Unsupervised Learning (K-Means + PCA) and Supervised Learning (Logistic Regression / SVM) within a single cohesive application.
2. **Zero Hallucination Guarantee:** Unlike generative chatbots, the assistant retrieves verified, faculty-approved syllabus definitions and formulas directly from master_ml_dataset.csv.
3. **Interactive No-Code Search Bar:** The built-in ipywidgets search bar allows students and evaluators to test unlimited questions interactively without modifying or re-running Python code.
4. **Computationally Lightweight:** Trains and executes in under 3 seconds on standard CPU hardware without requiring expensive GPUs or paid API keys.

7.2 Limitations

1. **Lexical Matching Sensitivity:** Because TF-IDF relies on bag-of-words lexical overlap, queries phrased with entirely unfamiliar synonyms not present in the 973-word vocabulary may yield lower cosine similarity scores.
2. **Fixed Corpus Size:** The current knowledge base is bounded to 644 curated ML questions and cannot answer out-of-domain queries (such as power systems or civil engineering) unless new CSV rows are added.
3. **Cluster Imbalance:** Natural topic distribution results in larger general clusters (Cluster 4 with 305 items and Cluster 0 with 229 items) compared to specialized mathematical clusters (Cluster 1 with 16 items).

8. Future Scope

1. **Dense Semantic Embeddings (Sentence-BERT):** Replacing TF-IDF vectors with pre-trained transformer embeddings (such as all-MiniLM-L6-v2) will enable deep semantic matching even when a student uses completely different wording.
2. **Retrieval-Augmented Generation (RAG) Synthesis:** Combining the top-3 retrieved answers with a local open-source language model (e.g., Llama-3 or Gemma) to synthesize conversational follow-up examples while remaining grounded in the syllabus.
3. **Standalone Web & Mobile Deployment:** Packaging the Python pipeline into a Streamlit or Gradio web application hosted on Hugging Face Spaces so students across the department can access the assistant from their smartphones.
4. **Multi-Subject Engineering Expansion:** Expanding master_ml_dataset.csv to cover core Electrical Engineering subjects such as Control Systems, Power Electronics, and Digital Signal Processing.

9. Conclusion

This project successfully designed, implemented, and evaluated an Intelligent Machine Learning Knowledge Assistant following all 10 stages of the Machine Learning Lifecycle. By merging and cleaning 644 domain-specific question-answer pairs into master_ml_dataset.csv and extracting 973 TF-IDF features, the system effectively bridged unstructured text processing with classical machine learning. Unsupervised K-Means clustering (k = 5, WCSS = 598.93) and 2D PCA projection automatically uncovered the latent thematic structure of the corpus without manual labeling. Supervised multi-class models trained on an 80:20 split demonstrated high predictive capability, with LinearSVC and Logistic Regression achieving 89.92% and 89.15% accuracy respectively on unseen test queries. Paired with a Cosine Similarity retrieval engine and an interactive ipywidgets search bar in Google Colab, the resulting system provides a fast, reliable, and practical study tool for engineering students.

10. References / Bibliography

[1] M. J. P. Hayes, "Machine Learning Questions Dataset," Hugging Face Datasets Repository, 2023. Available: https://huggingface.co/datasets/mjphayes/machine_learning_questions

[2] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research (JMLR), vol. 12, pp. 2825–2830, 2011. Available: https://scikit-learn.org/

[3] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. Cambridge, MA: MIT Press, 2016. Available: https://www.deeplearningbook.org/

[4] A. Thakur, Approaching (Almost) Any Machine Learning Problem. Abhishek Thakur, 2020, ch. 1–4, pp. 1–65.

[5] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," Information Processing & Management, vol. 24, no. 5, pp. 513–523, 1988.

[6] J. MacQueen, "Some methods for classification and analysis of multivariate observations," in Proc. 5th Berkeley Symp. Mathematical Statistics and Probability, vol. 1, 1967, pp. 281–297.

11. Appendices

Appendix A: Complete Project Source Code (ml_knowledge_assistant_project.py)

The complete, self-contained Python implementation used for data loading, preprocessing, TF-IDF feature engineering, K-Means clustering, PCA visualization, Logistic Regression classification, and the interactive Google Colab search bar is provided below:

```
#
================================================================================
# PROJECT: Machine Learning Knowledge Assistant
# CONCEPTS: Unsupervised Learning (K-Means, PCA) & Supervised Learning
(Logistic Regression)
#
================================================================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix
from sklearn.metrics.pairwise import cosine_similarity
import ipywidgets as widgets
from IPython.display import display, clear_output

# ------------------------------------------------------------------------------
--
# STEP 1 & 2: Problem Definition & Data Collection
# ------------------------------------------------------------------------------
--
df = pd.read_csv("master_ml_dataset.csv")
print("Step 2 - Total Dataset Rows Loaded:", len(df))

# ------------------------------------------------------------------------------
--
# STEP 3 & 4: Data Cleaning & Exploratory Data Analysis (EDA)
# ------------------------------------------------------------------------------
--
df = df.dropna(subset=["question", "answer"])
df = df.drop_duplicates(subset=["question"]).reset_index(drop=True)
print("Step 3 - Clean Dataset Rows:", len(df))
print("\nStep 4 - Dataset Sample:\n", df.head(3))

# ------------------------------------------------------------------------------
--
# STEP 5: Feature Engineering (Convert Text to Numbers using TF-IDF)
# ------------------------------------------------------------------------------
--
vectorizer = TfidfVectorizer(stop_words="english", max_features=1000)
X = vectorizer.fit_transform(df["question"])
print("\nStep 5 - TF-IDF Matrix Shape:", X.shape)
```

```
# ------------------------------------------------------------------------------
--
# STEP 6 & 7A: Unsupervised Learning (K-Means Clustering & PCA Plot)
# ------------------------------------------------------------------------------
--
kmeans = KMeans(n_clusters=5, random_state=42, n_init=10)
df["cluster"] = kmeans.fit_predict(X)

# 2D Visualization using PCA
pca = PCA(n_components=2, random_state=42)
X_pca = pca.fit_transform(X.toarray())

plt.figure(figsize=(7, 4))
sns.scatterplot(x=X_pca[:, 0], y=X_pca[:, 1], hue=df["cluster"],
palette="tab10")
plt.title("Unsupervised Learning: K-Means Clusters (2D PCA)")
plt.xlabel("PCA 1")
plt.ylabel("PCA 2")
plt.show()

# ------------------------------------------------------------------------------
--
# STEP 7B & 8: Supervised Learning (Classification & Evaluation)
# ------------------------------------------------------------------------------
--
y = df["cluster"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

classifier = LogisticRegression(max_iter=1000)
classifier.fit(X_train, y_train)

# Predict & Evaluate on Test Data
y_pred = classifier.predict(X_test)
print(f"\nStep 8 - Supervised Model Accuracy: {accuracy_score(y_test, y_pred)
* 100:.2f}%")
print("\nClassification Report:\n", classification_report(y_test, y_pred))

# Plot Confusion Matrix
plt.figure(figsize=(5, 4))
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt="d",
cmap="Blues")
plt.title("Supervised Learning: Confusion Matrix")
plt.xlabel("Predicted Topic")
plt.ylabel("Actual Topic")
plt.show()

# ------------------------------------------------------------------------------
--
# STEP 9 & 10: Deployment & Monitoring (Simple Interactive Search Bar)
# ------------------------------------------------------------------------------
--
search_box = widgets.Text(
    placeholder="Type your ML question here...",
    description="Question:",
    layout=widgets.Layout(width="60%")
)
search_btn = widgets.Button(description="Get Answer", button_style="primary")
output = widgets.Output()

def search_answer(_):
    with output:
```

```
        clear_output()
        query = search_box.value.strip()
        if not query:
            print("Please type a question first.")
            return

        # 1. Convert query to TF-IDF vector
        query_vec = vectorizer.transform([query])

        # 2. Supervised: Predict topic cluster
        pred_cluster = classifier.predict(query_vec)[0]

        # 3. Unsupervised: Find closest matching question using Cosine
Similarity
        scores = cosine_similarity(query_vec, X).flatten()
        best_index = np.argmax(scores)
        confidence = scores[best_index] * 100

        print("=" * 60)
        print("Your Question      :", query)
        print("Predicted Category : Cluster", pred_cluster, "(Supervised)")
        print(f"Match Confidence   : {confidence:.1f}% (Unsupervised)")
        print("Matched Question   :", df.loc[best_index, "question"])
        print("-" * 60)
        print("Answer:\n", df.loc[best_index, "answer"])
        print("=" * 60)

search_btn.on_click(search_answer)
search_box.on_submit(search_answer)

print("\nStep 9 - Search Bar Ready (Type a question below):")
display(widgets.HBox([search_box, search_btn]), output)
```